

智能公交安全助理

API 接口文档

Agent Service API

2026 年 2 月 7 日

目录

1	概述	3
1.1	简介	3
1.2	基础信息	3
1.3	版本信息	3
2	通用规范	3
2.1	请求格式	3
2.2	响应格式	3
2.3	HTTP 状态码	4
2.4	错误响应格式	4
3	接口列表	4
3.1	系统接口	4
3.1.1	健康检查	4
3.2	对话接口	5
3.2.1	获取会话列表	5
3.2.2	创建会话	5
3.2.3	获取会话详情	6
3.2.4	删除会话	7
3.3	对话接口	8
3.3.1	发送消息（非流式）	8
3.3.2	发送消息（流式）	10

4	数据模型	13
4.1	会话模型 (Session)	13
4.2	消息模型 (Message)	13
5	使用示例	14
5.1	完整流程示例	14
5.2	流式对话示例	15
6	错误处理	16
6.1	常见错误	16
6.2	错误响应示例	16
7	最佳实践	16
7.1	上下文管理	16
7.2	消息处理	17
7.3	错误处理	17
8	附录	17
8.1	工具说明	17
8.1.1	generate_driver_report	17
8.1.2	generate_vehicle_report	17
8.1.3	generate_route_report	17
8.1.4	consult_omni	18
8.2	变更日志	18

1 概述

1.1 简介

智能公交安全助理（Agent Service）提供基于大语言模型的智能对话服务，支持生成安全评估报告、智能咨询等功能。本文档描述了 Agent 服务的 REST API 接口规范。

1.2 基础信息

- 基础路径: /api/agent
- 协议: HTTP/HTTPS
- 数据格式: JSON
- 字符编码: UTF-8

1.3 版本信息

- API 版本: v1
- 文档版本: 1.0.0
- 最后更新: 2026 年 2 月 7 日

2 通用规范

2.1 请求格式

所有请求应包含以下 HTTP 头：

```
1 Content-Type: application/json
2 Accept: application/json
```

2.2 响应格式

所有响应均为 JSON 格式，包含以下字段：

- 成功响应: 直接返回数据对象或数组
- 错误响应: 包含 `error` 字段，描述错误信息

2.3 HTTP 状态码

状态码	说明
200	请求成功
201	创建成功
400	请求参数错误
404	资源不存在
500	服务器内部错误

表 1: HTTP 状态码说明

2.4 错误响应格式

```
1 {  
2   "error": "错误描述信息"  
3 }
```

3 接口列表

3.1 系统接口

3.1.1 健康检查

检查服务运行状态。

请求

```
1 GET /api/agent/health
```

响应

```
1 {  
2   "status": "ok",  
3   "timestamp": "2024-01-15T10:30:00.000Z"  
4 }
```

状态码: 200

3.2 对话接口

重要说明：Agent 服务采用智能路由机制，会根据用户消息的意图自动识别并分发到相应的工具进行处理。调用端无需关心具体的工具列表，只需发送消息即可。

3.2.1 获取会话列表

获取所有会话列表，按更新时间倒序排列。

请求

```
1 GET /api/agent/sessions
```

响应

```
1 [
2   {
3     "id": "session_123456",
4     "title": "新会话",
5     "preview": "请生成张三的安全报告...",
6     "updatedAt": "2024-01-15T10:30:00.000Z"
7   }
8 ]
```

字段说明

字段	类型	说明
id	String	会话 ID
title	String	会话标题
preview	String	会话预览（最后一条消息的摘要）
updatedAt	String	更新时间（ISO 8601 格式）

表 2: 会话列表响应字段

状态码: 200

3.2.2 创建会话

创建一个新的会话。

请求

```
1 POST /api/agent/sessions
2 Content-Type: application/json
```

```
3 {
4   "title": "新会话" // 可选，默认为"新会话"
5 }
6
```

请求参数

参数	类型	必填	说明
title	String	否	会话标题，默认为"新会话"

表 3: 创建会话请求参数

响应

```
1 {
2   "id": "session_123456",
3   "title": "新会话",
4   "preview": "",
5   "updatedAt": "2024-01-15T10:30:00.000Z"
6 }
```

状态码: 201

3.2.3 获取会话详情

获取指定会话的详细信息，包括所有消息记录。

请求

```
1 GET /api/agent/sessions/{sessionId}
```

路径参数

参数	类型	说明
sessionId	String	会话 ID

表 4: 获取会话详情路径参数

响应

```
1 {
2   "id": "session_123456",
3   "title": "新会话",
```

```
4  "preview": "请生成张三的安全报告...",
5  "updatedAt": "2024-01-15T10:30:00.000Z",
6  "messages": [
7    {
8      "id": "message_001",
9      "role": "user",
10     "content": "请生成张三的安全报告",
11     "createdAt": "2024-01-15T10:25:00.000Z",
12     "status": "complete"
13   },
14   {
15     "id": "message_002",
16     "role": "assistant",
17     "content": "正在为您生成张三的安全报告...",
18     "createdAt": "2024-01-15T10:25:05.000Z",
19     "status": "complete",
20     "metadata": {
21       "tool": "generate_driver_report",
22       "args": {
23         "driver_name": "张三"
24       }
25     }
26   }
27 ]
28 }
```

字段说明

状态码: 200

错误响应: 404（会话不存在）

3.2.4 删除会话

删除指定的会话及其所有消息记录。

请求

```
1 DELETE /api/agent/sessions/{sessionId}
```

路径参数

响应

字段	类型	说明
id	String	会话 ID
title	String	会话标题
preview	String	会话预览
updatedAt	String	更新时间
messages	Array	消息列表
messages[].id	String	消息 ID
messages[].role	String	消息角色（user/assistant/system/tool）
messages[].content	String	消息内容
messages[].createdAt	String	创建时间
messages[].status	String	状态（complete/error）
messages[].metadata	Object	元数据（可选）

表 5: 会话详情响应字段

参数	类型	说明
sessionId	String	会话 ID

表 6: 删除会话路径参数

```
1 {
2   "success": true
3 }
```

状态码: 200
错误响应: 404（会话不存在）

3.3 对话接口

3.3.1 发送消息（非流式）

向 Agent 发送消息，并获取 AI 助手的回复。此接口会等待完整响应后返回。
请求

```
1 POST /api/agent/chat
2 Content-Type: application/json
3
4 {
5   "content": "请生成张三的安全报告",
```



```
6  "messages": [  
7    {  
8      "role": "user",  
9      "content": "你好"  
10   },  
11   {  
12     "role": "assistant",  
13     "content": "您好，我是智能公交安全助理，有什么可以帮助您的吗？"  
14   }  
15 ]  
16 }
```

请求参数

参数	类型	必填	说明
content	String	是	当前用户消息内容
messages	Array	否	历史消息列表，用于构建上下文。每个消息包含 role（user/assistant）和

表 7: 发送消息请求参数

响应

```
1  {  
2    "id": "message_002",  
3    "role": "assistant",  
4    "content": "正在为您生成张三的安全报告...",  
5    "createdAt": "2024-01-15T10:25:05.000Z",  
6    "status": "complete",  
7    "metadata": {  
8      "tool": "generate_driver_report",  
9      "args": {  
10       "driver_name": "张三"  
11     },  
12     "iterations": 2  
13   }  
14 }
```

字段说明

状态码: 200

错误响应

字段	类型	说明
id	String	消息 ID
role	String	消息角色（assistant）
content	String	AI 回复内容
createdAt	String	创建时间
status	String	状态（complete/error）
metadata	Object	元数据（可选）
metadata.tool	String	使用的工具名称
metadata.args	Object	工具参数
metadata.iterations	Number	工具调用迭代次数

表 8: 发送消息响应字段

- 400: 缺少 sessionId 或 content
- 404: 会话不存在

3.3.2 发送消息（流式）

向指定会话发送消息，以 Server-Sent Events (SSE) 流式返回 AI 助手的回复。适用于需要实时显示回复内容的场景。

请求

```
1 POST /api/agent/chat/stream
2 Content-Type: application/json
3
4 {
5   "sessionId": "session_123456",
6   "content": "请分析一下最近的风险趋势"
7 }
```

请求参数

参数	类型	必填	说明
sessionId	String	是	会话 ID
content	String	是	用户消息内容

表 9: 流式发送消息请求参数

响应格式

响应为 Server-Sent Events (SSE) 流，包含以下事件类型：

- 1. **start**: 开始处理
- 2. **chunk**: 内容片段（可能多次）
- 3. **complete**: 完成处理
- 4. **error**: 错误信息（如果发生错误）

事件格式

```
1 # 开始事件
2 data: {"type": "start", "messageId": "message_002"}
3
4 # 内容片段事件（可能多次）
5 data: {"type": "chunk", "content": "正在"}
6
7 data: {"type": "chunk", "content": "为您"}
8
9 data: {"type": "chunk", "content": "生成"}
10
11 # 完成事件
12 data: {"type": "complete", "messageId": "message_002", "metadata": {...}}
13
14 # 错误事件（如果发生）
15 data: {"type": "error", "error": "错误信息"}
```

事件字段说明

字段	类型	说明
type	String	事件类型（start/chunk/complete/error）
messageId	String	消息 ID（start/complete 事件）
content	String	内容片段（chunk 事件）
metadata	Object	元数据（complete 事件）
error	String	错误信息（error 事件）

表 10: SSE 事件字段说明

HTTP 头

```
1 Content-Type: text/event-stream
2 Cache-Control: no-cache
3 Connection: keep-alive
```

状态码: 200

错误响应: 404 (会话不存在)

客户端示例 (JavaScript)

```
1 const response = await fetch('/api/agent/chat/stream', {
2   method: 'POST',
3   headers: { 'Content-Type': 'application/json' },
4   body: JSON.stringify({
5     sessionId: 'session_123456',
6     content: '请分析一下最近的风险趋势'
7   })
8 });
9
10 const reader = response.body.getReader();
11 const decoder = new TextDecoder();
12
13 while (true) {
14   const { done, value } = await reader.read();
15   if (done) break;
16
17   const chunk = decoder.decode(value);
18   const lines = chunk.split('\n');
19
20   for (const line of lines) {
21     if (line.startsWith('data: ')) {
22       const data = JSON.parse(line.slice(6));
23
24       switch (data.type) {
25         case 'start':
26           console.log('开始处理:', data.messageId);
27           break;
28         case 'chunk':
29           console.log('内容片段:', data.content);
30           // 实时显示内容
```

```
31         break;
32     case 'complete':
33         console.log('处理完成:', data.metadata);
34         break;
35     case 'error':
36         console.error('错误:', data.error);
37         break;
38     }
39 }
40 }
41 }
```

4 数据模型

4.1 会话模型 (Session)

```
1 {
2     "id": "string",           // 会话 ID
3     "title": "string",       // 会话标题
4     "preview": "string",     // 会话预览
5     "updatedAt": "string"    // 更新时间 (ISO 8601)
6 }
```

4.2 消息模型 (Message)

```
1 {
2     "id": "string",           // 消息 ID
3     "role": "string",         // 角色: user/assistant/system/tool
4     "content": "string",      // 消息内容
5     "createdAt": "string",    // 创建时间 (ISO 8601)
6     "status": "string",       // 状态: complete/error
7     "metadata": {             // 元数据 (可选)
8         "tool": "string",     // 使用的工具名称
9         "args": {},           // 工具参数
10        "iterations": 0        // 工具调用迭代次数
11    }
12 }
```

5 使用示例

5.1 完整流程示例

以下示例展示如何创建一个会话并进行对话：

```
1 // 1. 创建会话
2 const createResponse = await fetch('/api/agent/sessions', {
3   method: 'POST',
4   headers: { 'Content-Type': 'application/json' },
5   body: JSON.stringify({ title: '安全报告生成' })
6 });
7 const session = await createResponse.json();
8 console.log('会话ID:', session.id);
9
10 // 2. 发送消息
11 const chatResponse = await fetch('/api/agent/chat', {
12   method: 'POST',
13   headers: { 'Content-Type': 'application/json' },
14   body: JSON.stringify({
15     sessionId: session.id,
16     content: '请生成张三的安全报告'
17   })
18 });
19 const message = await chatResponse.json();
20 console.log('AI 回复:', message.content);
21
22 // 3. 获取会话详情
23 const detailResponse = await fetch(`/api/agent/sessions/${session.id}`);
24 const detail = await detailResponse.json();
25 console.log('会话消息数:', detail.messages.length);
26
27 // 4. 删除会话
28 await fetch(`/api/agent/sessions/${session.id}`, {
29   method: 'DELETE'
```

```
30 });
```

5.2 流式对话示例

```
1  async function streamChat(content, messages = []) {
2      const response = await fetch('/api/agent/chat/stream', {
3          method: 'POST',
4          headers: { 'Content-Type': 'application/json' },
5          body: JSON.stringify({ content, messages })
6      });
7
8      const reader = response.body.getReader();
9      const decoder = new TextDecoder();
10     let fullContent = '';
11
12     while (true) {
13         const { done, value } = await reader.read();
14         if (done) break;
15
16         const chunk = decoder.decode(value);
17         const lines = chunk.split('\n');
18
19         for (const line of lines) {
20             if (line.startsWith('data: ')) {
21                 const data = JSON.parse(line.slice(6));
22
23                 if (data.type === 'chunk') {
24                     fullContent += data.content;
25                     // 实时更新UI
26                     updateUI(fullContent);
27                 } else if (data.type === 'complete') {
28                     console.log('完成, 消息ID:', data.messageId);
29                 } else if (data.type === 'error') {
30                     console.error('错误:', data.error);
31                 }
32             }
33         }
34     }
```

```
35  
36     return fullContent;  
37 }
```

6 错误处理

6.1 常见错误

错误码	错误信息	解决方案
400	缺少 sessionId 或 content	检查请求参数
404	会话不存在	确认会话 ID 是否正确
500	服务器内部错误	联系技术支持

表 11: 常见错误及解决方案

6.2 错误响应示例

```
1 {  
2   "error": "会话不存在"  
3 }
```

7 最佳实践

7.1 上下文管理

- Agent 服务采用无状态设计，不提供会话管理功能
- 调用端需要自行管理会话和消息历史
- 每次请求时传递完整的历史消息列表（messages 参数）
- 建议限制历史消息数量，避免超出上下文窗口（默认保留最近 50 条消息）
- 历史消息格式：每个消息包含 role（user/assistant）和 content 字段

7.2 消息处理

- 对于需要实时反馈的场景，使用流式接口（`/chat/stream`）
- 对于批量处理或不需要实时反馈的场景，使用非流式接口（`/chat`）
- 合理控制消息长度，避免超出上下文窗口限制

7.3 错误处理

- 始终检查 HTTP 状态码
- 实现重试机制，处理网络错误
- 记录错误日志，便于问题排查

8 附录

8.1 工具说明

8.1.1 generate_driver_report

生成驾驶员安全评估报告，包含风险评分、行为画像及改进建议。

参数

- `driver_name` (String, 必填): 驾驶员姓名

8.1.2 generate_vehicle_report

生成车辆运行分析报告，分析车辆的维修记录、故障率及能耗数据。

参数

- `vehicle_id` (String, 必填): 车辆 ID 或车牌号

8.1.3 generate_route_report

生成线路风险扫描报告，基于线路的历史事故数据和路况信息。

参数

- `route_name` (String, 必填): 线路名称或编号

8.1.4 consult_omni

智能咨询助手，支持趋势分析、归因分析、政策问答等复杂查询。

参数

- query (String, 必填): 用户查询内容

8.2 变更日志

版本	日期	变更说明
1.0.0	2024-01-15	初始版本

表 12: API 版本变更日志